



UNIVERSIDADE FEDERAL DE ALAGOAS - UFAL
INSTITUTO DE COMPUTAÇÃO

Relatório de Projeto Acadêmico

ROMA QUIZ: jogo em formato de quiz educacional com arquitetura de microserviços.

Quiz online sobre o Império Romano com frontend Next.js, microserviços Express, inteligência artificial e banco PostgreSQL.

Alunos: Caio Mascarenhas Soares

Link do GitHub: <https://github.com/CaioMascarenhas/quiz-imperio-romano>

Link do vídeo demonstrativo: <https://youtu.be/sXvT0QiAKIM>

Maceió - AL
2026

SUMÁRIO

1. Enunciado	3
2. Product Backlog	3
2.1. Sprint 1 - Infraestrutura Base	3
2.2. Sprint 2 - Lógica do Quiz	4
2.3. Sprint 3 - Interface e Experiência	4
3. Sprint Backlog	4
4. Arquitetura do Sistema	5
5. Diagrama de Componentes	6
6. Tecnologias Utilizadas	7
7. Microserviços e Serviços de Apoio	8
8. Funcionamento do Sistema	10
9. Funcionamento da Inteligência Artificial	12
10. API Endpoints	13
11. Testes e Validação	14
12. Links do Projeto	15
13. Considerações Finais	15
14. Conclusão	16

1. ENUNCIADO

O projeto Roma Quiz consiste em uma aplicação web educacional voltada ao estudo do Império Romano. A proposta é oferecer ao usuário uma experiência interativa de aprendizagem por meio de quizzes temáticos, chat com inteligência artificial, ranking de pontuação e histórico de respostas.

A aplicação foi desenvolvida com uma arquitetura distribuída baseada em microserviços. O frontend é responsável pela experiência visual e comunicação com as APIs, enquanto os serviços de backend cuidam separadamente da autenticação, geração de questões, chat, ranking e persistência de tentativas.

O sistema utiliza Docker Compose para orquestrar os serviços e PostgreSQL como banco de dados compartilhado. A inteligência artificial é usada no serviço de quiz para gerar perguntas educativas e respostas de chat sobre o Império Romano.

2. PRODUCT BACKLOG

Item	Descrição	Prioridade
Cadastro e login	Permitir criação de conta, autenticação do usuário e emissão de token JWT.	Alta
Quiz por tema	Gerar perguntas sobre temas do Império Romano usando IA e retornar alternativas, resposta correta e explicações.	Alta
Histórico	Salvar tentativas respondidas pelo usuário e exibir respostas escolhidas, corretas e explicações.	Alta
Ranking	Exibir usuários ordenados por quantidade de acertos, com títulos de acordo com a colocação.	Média
Chat educativo	Permitir perguntas abertas sobre o Império Romano usando IA com contexto recente da conversa.	Média
Dockerização	Executar frontend, backends e banco por meio de Docker Compose.	Alta

2.1. Sprint 1 - Infraestrutura Base

A primeira sprint concentrou-se na criação da estrutura do projeto, separando frontend, backends e banco de dados. Também foram definidos os containers, as portas de execução e as variáveis de ambiente necessárias para o funcionamento local.

2.2. Sprint 2 - Lógica do Quiz

A segunda sprint implementou a integração com a IA para geração das questões, o formato JSON retornado pelo serviço de quiz e a gravação de tentativas no banco de dados. Também foram criadas as tabelas de usuários e respostas.

2.3. Sprint 3 - Interface e Experiência

A terceira sprint concentrou-se na interface Next.js, com páginas de landing page, login, cadastro, dashboard, quiz, chat, ranking e histórico. Foram adicionadas imagens relacionadas ao Império Romano e animação Lottie durante a geração das questões.

3. SPRINT BACKLOG

Sprint	Tarefas	Status
Sprint 1	Configurar Docker Compose, PostgreSQL, estrutura dos microserviços e rotas de saúde.	Concluído
Sprint 2	Implementar autenticação JWT, geração de perguntas com IA e persistência das tentativas.	Concluído
Sprint 3	Construir telas do usuário, ranking, histórico, chat e integração do frontend com as APIs.	Concluído

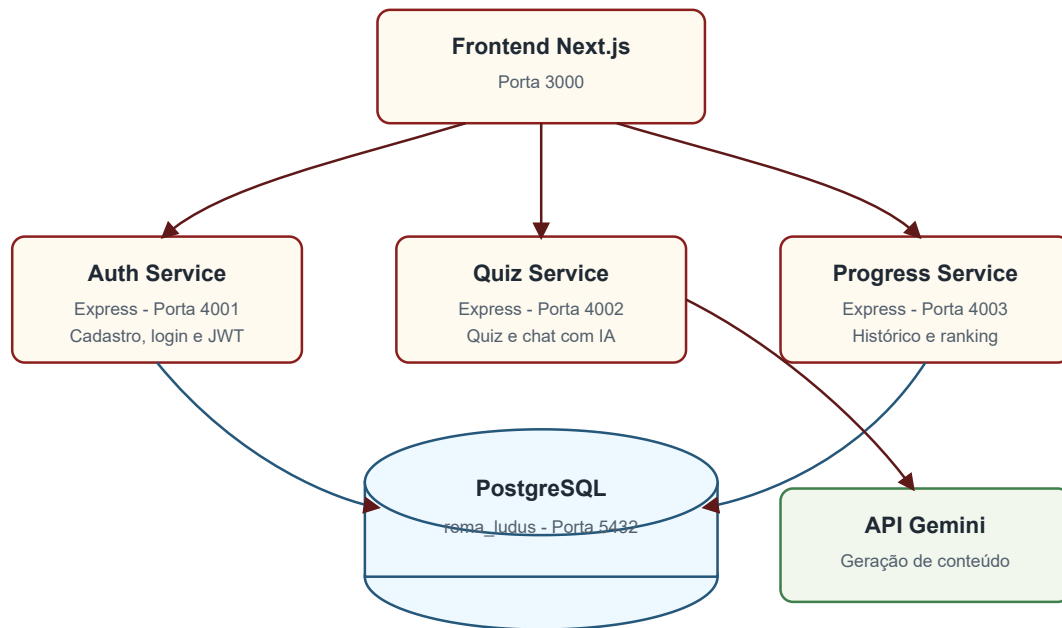
4. ARQUITETURA DO SISTEMA

A arquitetura do Roma Quiz é organizada em serviços independentes. O frontend Next.js consome os microserviços por HTTP. O `auth-service` gerencia cadastro, login e validação de sessão. O `quiz-service` centraliza a geração de perguntas e respostas de chat por meio da API Gemini. O `progress-service` registra tentativas, consulta histórico e calcula o ranking.

O banco PostgreSQL é compartilhado entre os serviços que precisam de persistência. Essa decisão simplifica a execução acadêmica do projeto, mantendo a separação lógica dos domínios por serviço.

Características da arquitetura: separação por responsabilidade, comunicação HTTP/JSON, autenticação por JWT, persistência em PostgreSQL, orquestração com Docker Compose e integração com IA para conteúdo educacional.

5. DIAGRAMA DE COMPONENTES



Comunicação principal: HTTP/JSON com token JWT nas rotas protegidas

Figura 1 - Diagrama de componentes dos microserviços do Roma Quiz.

5.1. Execução dos containers

A execução local do projeto pode ser observada no Docker Desktop, com os containers do frontend, dos três microserviços de backend e do banco PostgreSQL ativos. Essa visualização confirma as portas expostas e a separação dos componentes em containers independentes.

Containers [Give feedback](#)

Container CPU usage ⓘ
0.00% / 800% (8 CPUs available)

Container memory usage ⓘ
177.95MB / 7.53GB [Show charts](#)

🔍 Search

☒ Only show running containers

Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions
quizimperiormano	-	-	-	0%	23 minutes ago	
roma-frontend	baaabc9dc28	quizimperiormano-frontend	3000:3000	0%	23 minutes ago	
roma-auth-service	0d6ccb9d1253	quizimperiormano-auth-service	4001:4001	0%	23 minutes ago	
roma-progress-service	df09276e7e2a	quizimperiormano-progress-service	4003:4003	0%	23 minutes ago	
roma-postgres	20fd565a822e	postgres:16-alpine	5432:5432	0%	23 minutes ago	
roma-quiz-service	407ef59a9272	quizimperiormano-quiz-service	4002:4002	0%	23 minutes ago	

Figura 2 - Containers do Roma Quiz em execução no Docker Desktop.

6. TECNOLOGIAS UTILIZADAS

Backend

Os microserviços foram implementados com Node.js, Express, CORS, Dotenv, JSON Web Token e PostgreSQL. O serviço de autenticação utiliza BcryptJS para hash de senhas. O serviço de quiz utiliza @google/genai para integração com a API Gemini.

Frontend

O frontend utiliza Next.js 16, React 19, TypeScript, Tailwind CSS, Radix UI, componentes reutilizáveis, Lucide React, Lottie React e bibliotecas auxiliares para construção da interface.

DevOps

O ambiente é executado por Docker Compose, com containers separados para frontend, autenticação, quiz, progresso e PostgreSQL.

Arquitetura

A aplicação segue uma arquitetura orientada a microserviços, com comunicação HTTP, payloads JSON, autenticação por token e persistência relacional.

7. MICROSERVIÇOS E SERVIÇOS DE APOIO

O sistema possui três microserviços de backend. O frontend e o PostgreSQL aparecem como serviços no Docker Compose, mas não são classificados como microserviços: o frontend é a interface de usuário e o PostgreSQL é a infraestrutura de persistência.

7.1. Auth Service - Porta 4001

Responsável por cadastro, login, hash de senha e geração de token JWT. Também possui rota para validar a sessão atual do usuário. O serviço cria a tabela `users` no PostgreSQL quando iniciado.

7.2. Quiz Service - Porta 4002

Responsável por gerar quizzes e responder ao chat educacional. O serviço valida o JWT nas rotas protegidas e consulta a API Gemini para produzir perguntas, alternativas, feedbacks e explicações sobre o Império Romano.

7.3. Progress Service - Porta 4003

Responsável por registrar tentativas de quiz, consultar o histórico individual e gerar ranking dos usuários. O serviço cria a tabela `quiz_attempts` e relaciona cada tentativa ao usuário autenticado.

7.4. Frontend Next.js - Porta 3000

Serviço de apoio responsável pela interface do usuário. Inclui landing page, páginas de login e cadastro, dashboard, tela de quiz, chat, ranking e histórico. O frontend armazena o token no navegador e chama os microserviços por meio das URLs públicas configuradas no Docker Compose.

7.5. PostgreSQL - Porta 5432

Banco de dados relacional utilizado para persistir usuários e tentativas. A base configurada no Docker Compose chama-se `roma_ludus`.

8. FUNCIONAMENTO DO SISTEMA

8.1. Fluxo de Autenticação

O usuário cria uma conta informando nome, e-mail e senha. O `auth-service` aplica hash na senha, grava o usuário no banco e retorna um token JWT. Esse token é enviado nas chamadas protegidas para quiz, chat, ranking e histórico.

8.2. Tela de Login e Registro

As telas de login e registro fazem a comunicação com o serviço de autenticação. Após o sucesso, o usuário é direcionado para a área principal da aplicação.

8.3. Geração de Quiz

O usuário escolhe um tema relacionado ao Império Romano. O frontend envia o tema para `/quiz/generate`. O serviço monta um prompt, solicita a geração à IA e retorna cinco questões em JSON, cada uma com enunciado, alternativas, resposta correta, explicação e feedbacks.

8.4. Registro de Respostas

Ao responder uma questão, o frontend envia a tentativa ao `progress-service`. A tentativa fica associada ao usuário autenticado, permitindo montar histórico e ranking.

8.5. Chat Educativo

O chat usa o `quiz-service` para responder perguntas em português do Brasil. O serviço envia até cinco mensagens recentes como contexto, mantendo continuidade na conversa.

8.6. Ranking e Histórico

O ranking ordena usuários por acertos, enquanto o histórico lista as tentativas do usuário com tema, pergunta, resposta escolhida, resposta correta e explicação.

9. FUNCIONAMENTO DA INTELIGÊNCIA ARTIFICIAL

A inteligência artificial é um dos elementos centrais do Roma Quiz. Ela é utilizada no `quiz-service`, microserviço executado na porta `4002`, para transformar temas escolhidos pelo usuário em perguntas educativas e para responder dúvidas abertas no chat sobre o Império Romano.

9.1. Integração com o Gemini

O serviço de quiz utiliza a biblioteca `@google/genai` para consultar a API Gemini. A chave de acesso é lida pelas variáveis de ambiente `GEMINI_API_KEY` ou `GOOGLE_API_KEY`, e o modelo utilizado é definido em `GEMINI_MODEL`. Dessa forma, o código não deixa credenciais fixas no repositório e permite trocar o modelo sem alterar a implementação.

9.2. Geração de perguntas do quiz

Quando o usuário seleciona um tema, o frontend envia uma requisição protegida por JWT para `/quiz/generate`. O backend monta um prompt pedindo cinco questões sobre o tema, cada uma com enunciado, quatro alternativas, índice da resposta correta, explicação, feedbacks para alternativas erradas e uma imagem local opcional. A IA deve responder apenas em JSON válido, seguindo o formato esperado pela aplicação.

1. O usuário escolhe o tema no frontend.
2. O frontend envia o tema e o token de autenticação para o `quiz-service`.
3. O serviço cria o prompt com as regras do quiz e o catálogo de imagens disponíveis.
4. A API Gemini gera o conteúdo educacional.
5. O backend extrai e valida o JSON retornado antes de responder ao frontend.
6. O frontend renderiza as questões, alternativas, imagem relacionada e feedbacks.

9.3. Catálogo de imagens e contextualização

O prompt enviado à IA inclui uma lista de imagens locais disponíveis no frontend, como Coliseu, Senado, Júlio César, Pax Romana, legiões e queda romana. Com isso, a IA pode escolher uma imagem coerente com o assunto da pergunta, mas sem depender de links externos. Essa estratégia melhora a experiência visual e mantém os recursos controlados dentro do próprio projeto.

9.4. Chat educativo com contexto recente

Além do quiz, o endpoint `/chat` permite conversar com um tutor virtual de história. O frontend envia a pergunta atual e até cinco mensagens recentes da conversa. O backend transforma esse histórico em contexto textual, solicita uma resposta em português do Brasil, com tom educativo e limite de até 140 palavras, e retorna a resposta ao usuário.

9.5. Tratamento de falhas e validação

Como respostas de IA podem variar, o backend possui uma função para remover blocos de código, localizar o primeiro objeto JSON retornado e fazer o parse do conteúdo. Se a IA não responder, retornar texto vazio ou produzir JSON inválido, o serviço registra o erro nos logs e envia ao frontend uma mensagem amigável informando que não foi possível gerar o conteúdo naquele momento.

Recurso de IA	Endpoint	Entrada principal	Saída esperada
Geração de quiz	/quiz/generate	Tema escolhido pelo usuário e token JWT.	Cinco questões em JSON com alternativas, resposta correta, explicação, feedbacks e imagem opcional.
Chat educativo	/chat	Pergunta atual, histórico recente e token JWT.	Resposta curta, contextualizada e educativa sobre o Império Romano.

10. API ENDPOINTS

Serviço	Endpoint	Método	Finalidade
Auth	/health	GET	Verifica se o serviço está ativo.
Auth	/register	POST	Cadastra usuário e retorna token JWT.
Auth	/login	POST	Autentica usuário existente.
Auth	/me	GET	Valida o token e retorna dados do usuário.
Quiz	/quiz/generate	POST	Gera questões por tema usando IA.
Quiz	/chat	POST	Responde perguntas educacionais sobre o Império Romano.
Progress	/attempts	POST	Registra tentativa respondida pelo usuário.
Progress	/ranking	GET	Consulta ranking geral ou por tema.
Progress	/history	GET	Consulta histórico do usuário autenticado.

10.1. Exemplos de uso das rotas

Os exemplos abaixo consideram a aplicação executando localmente pelo Docker Compose. Nas rotas protegidas, o valor `SEU_TOKEN_JWT` deve ser substituído pelo token

retornado no cadastro ou login.

Cadastro de usuário - Auth Service

```
curl -X POST http://localhost:4001/register ^  
-H "Content-Type: application/json" ^  
-d "{\"name\":\"Caio Mascarenhas\", \"email\":\"caio@example.com\", \"password\":\"123456\"}"
```

Login - Auth Service

```
curl -X POST http://localhost:4001/login ^  
-H "Content-Type: application/json" ^  
-d "{\"email\":\"caio@example.com\", \"password\":\"123456\"}"
```

Consulta do usuário autenticado - Auth Service

```
curl http://localhost:4001/me ^  
-H "Authorization: Bearer SEU_TOKEN_JWT"
```

Geração de quiz - Quiz Service

```
curl -X POST http://localhost:4002/quiz/generate ^  
-H "Content-Type: application/json" ^  
-H "Authorization: Bearer SEU_TOKEN_JWT" ^  
-d "{\"theme\":\"Imperadores Romanos\"}"
```

Chat educativo - Quiz Service

```
curl -X POST http://localhost:4002/chat ^  
-H "Content-Type: application/json" ^  
-H "Authorization: Bearer SEU_TOKEN_JWT" ^  
-d "{\"message\":\"Explique a Pax Romana.\", \"history\":[]}"
```

Registro de tentativa - Progress Service

```
curl -X POST http://localhost:4003/attempts ^  
-H "Content-Type: application/json" ^  
-H "Authorization: Bearer SEU_TOKEN_JWT" ^  
-d "{\"theme\":\"Imperadores Romanos\", \"question\":\"Quem foi Augusto?\", \"selectedAnswer\":\"Primeiro imperador romano\", \"correctAnswer\":\"Primeiro imperador romano\", \"isCorrect\":true, \"explanation\":\"Augusto inaugurou o Principado.\"}"
```

Ranking e histórico - Progress Service

```
curl http://localhost:4003/ranking ^  
-H "Authorization: Bearer SEU_TOKEN_JWT"  
  
curl "http://localhost:4003/history?theme=Imperadores%20Romanos" ^  
-H "Authorization: Bearer SEU_TOKEN_JWT"
```

11. TESTES E VALIDAÇÃO

A validação do projeto pode ser realizada com execução local via Docker Compose e testes manuais das principais funcionalidades. Também é possível validar as rotas de saúde de cada microserviço antes de testar os fluxos completos.

11.1. Testes Manuais Realizados

- Criação de conta com nome, e-mail e senha.
- Login e armazenamento do token de sessão.
- Geração de quiz por tema com cinco questões.
- Envio de pergunta ao chat educativo.
- Registro de tentativa após resposta do usuário.
- Consulta de ranking e histórico.

11.2. Comandos de Validação

Para executar o projeto completo:

```
docker compose up --build
```

Após a inicialização, acessar `http://localhost:3000`. As APIs ficam disponíveis em `4001`, `4002` e `4003`.

12. LINKS DO PROJETO

Repositório GitHub do projeto:

```
https://github.com/CaioMascarenhas/quiz-imperio-romano
```

Vídeo demonstrativo no YouTube:

```
https://youtu.be/sXvT0QiAKIM
```

13. CONSIDERAÇÕES FINAIS

13.1. Objetivos Alcançados

O projeto alcançou o objetivo de construir uma aplicação educacional interativa sobre o Império Romano, com autenticação, geração de quiz por IA, chat, ranking, histórico e execução em containers.

13.2. Aprendizados

O desenvolvimento permitiu aplicar conceitos de microserviços, APIs REST, autenticação com JWT, persistência relacional, integração com IA generativa e construção de interfaces modernas com Next.js.

13.3. Melhorias Futuras

- Adicionar testes automatizados de integração para as APIs.
- Separar bancos de dados por microserviço para maior independência.
- Persistir conversas do chat para consulta posterior.
- Adicionar painel administrativo para curadoria de temas e imagens.
- Implementar observabilidade com logs estruturados e métricas.

13.4. Desafios Enfrentados

Entre os principais desafios estão a padronização do JSON gerado pela IA, a validação de sessão entre serviços independentes e a organização do ambiente Docker para que frontend, backends e banco funcionem de forma integrada.

14. CONCLUSÃO

O Roma Quiz demonstra a aplicação prática de uma arquitetura baseada em microserviços em um contexto educacional. A separação entre autenticação, quiz, progresso e interface torna o sistema mais organizado e facilita a manutenção. O uso de IA amplia a dinâmica do aprendizado ao permitir geração de questões e respostas contextualizadas sobre o Império Romano.

Assim, o projeto atende aos requisitos de uma aplicação web completa, com interface, backend, persistência, autenticação, componentes distribuídos e documentação arquitetural.